Experiment No: 1

Student Name: Prionti Das
UID: 24BAI71005
Branch: AIT-CSE
Section/Group: 24AIT_NTPP-3[B]
Semester: 5TH
Date of Performance:
Subject Name: Competitive Coding – II
Subject Code: 24CSP-305

Experiment 1.1

Aim: To implement a C++ program to determine whether an array contains duplicate elements whose indices are at most k positions apart.

Objective:

- To understand the use of unordered_map.
- To store and track the latest index of each element.
- To efficiently detect nearby duplicate elements.

Problem Statement / LeetCode Link:

https://leetcode.com/problems/contains-duplicate-ii/description/

Given an integer array nums and an integer k, determine whether there are two distinct indices i and j such that:

- nums[i] == nums[j]
- |i - j| <= k

Return true if such a pair exists; otherwise, return false.

Algorithm:

1. Create an `unordered_map` named `lastIndex` to store each element's most recent index.
2. Traverse the array from left to right.
3. For each element:
  - Check if it already exists in `lastIndex`.
  - If it exists, calculate the difference between the current index and its previous index.
  - If the difference is less than or equal to `k`, return `true`.

4. Update the element's index with the current index.
5. If no nearby duplicate is found, return `false`.

Code:

```
#include <unordered_map>
#include <vector>
using namespace std;

class Solution {
public:
   bool containsNearbyDuplicate(vector<int>& nums, int k) {
      unordered_set<int> window;

      for (int i = 0; i < nums.size(); ++i) {

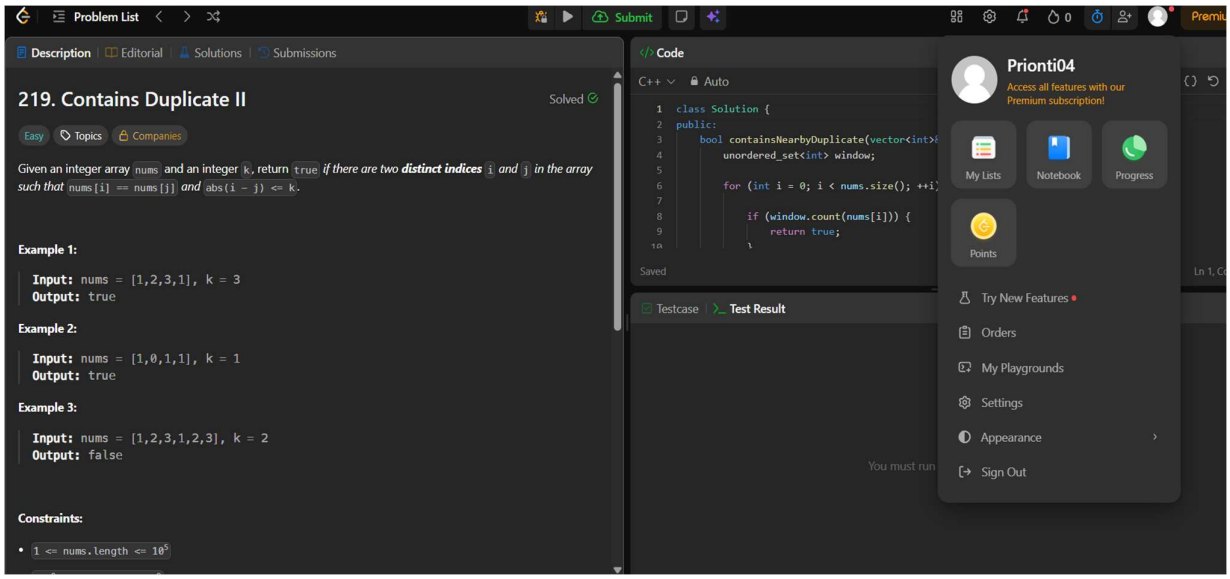
         if (window.count(nums[i])) {
            return true;
         }
```

```
            window.insert(nums[i]);


            if (window.size() > k) {
                window.erase(nums[i - k]);
            }
        }

        return false;
    }
};
```

Observations/Outcome: The program successfully determines whether the array contains duplicate elements within a distance of k indices.



Time Complexity: **O(n)** average, where n is the number of elements in the array.

Space Complexity: **O(n)** in the worst case, due to the unordered_map.

Learning outcomes:

1. Learned how to use unordered_map for fast searching and indexing.
2. Understood how to detect duplicates efficiently.
3. Learned to reduce the time complexity from **O(n²)** to **O(n)** using hashing.
4. Improved understanding of index-based array problems.

Experiment 1.2

Aim: To implement a C++ program to find the product of all elements of an array except the element at the current index.

Objective:

- To calculate the required products efficiently.
- To understand the use of prefix and suffix products.
- To solve the problem without using division.

Problem Statement / LeetCode Link:

https://leetcode.com/problems/product-of-array-except-self/submissions/2143248311/

Given an integer array nums, return an array answer such that answer[i] is equal to the product of all elements of nums except nums[i].

Algorithm:

- Create an answer array of size n initialized with 1.
- Traverse the array from left to right:
  - Store the product of all elements before the current index in answer[i].
  - Update the left product by multiplying it with nums[i].
- Traverse the array from right to left:
  - Multiply answer[i] by the product of all elements after the current index.
  - Update the right product by multiplying it with nums[i].
- Return the answer array.

Code:

```
#include <vector>
using namespace std;

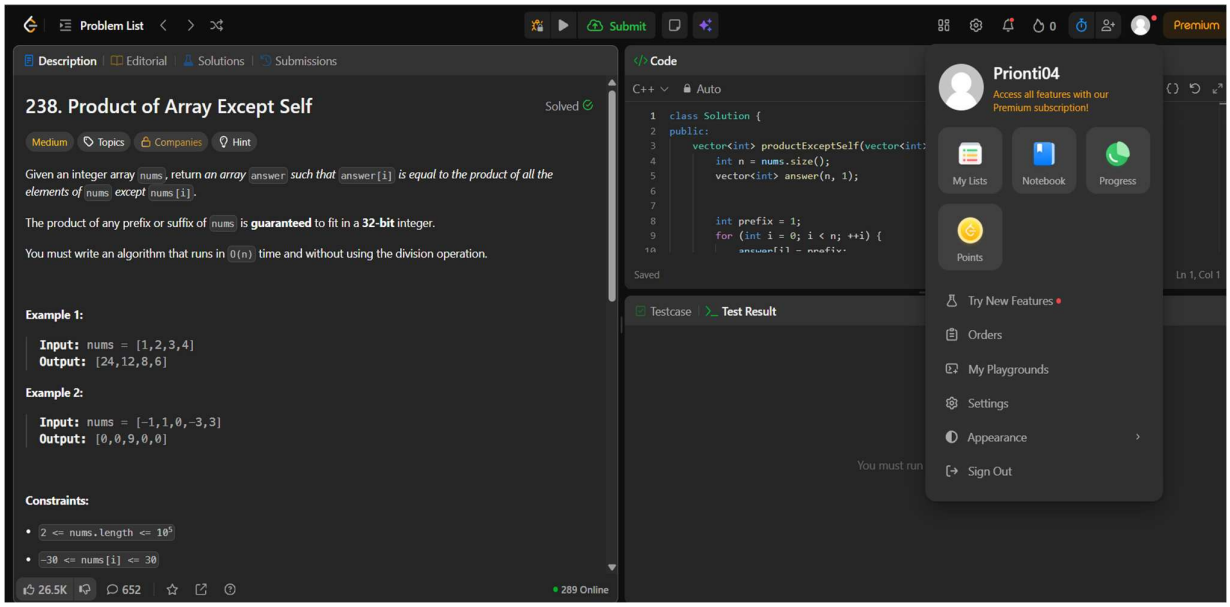
class Solution {
public:
    vector<int> productExceptSelf(vector<int>& nums) {
        int n = nums.size();
        vector<int> answer(n, 1);


        int prefix = 1;
        for (int i = 0; i < n; ++i) {
            answer[i] = prefix;
            prefix *= nums[i];
        }


        int suffix = 1;
        for (int i = n - 1; i >= 0; --i) {
            answer[i] *= suffix;
            suffix *= nums[i];
        }

        return answer;
    }
};
```

Observations/Outcome: The program successfully calculates the product of all array elements except the element at each corresponding index without using division.



Time Complexity: O(n) The array is traversed twice.

Space Complexity: O(1) auxiliary space, excluding the output array.

Learning outcomes:

1. Learned how to use **prefix and suffix products**.
2. Learned to solve the problem efficiently without division.
3. Understood how to reduce the time complexity to **O(n)**.
4. Improved understanding of array traversal and space optimization.